

《编译原理》课程设计报告

25—26 学年 第二学期

学院（系）： 计算机科学与工程学院

班 级： 123030701

学 号： 12303070250

姓 名： 黄彬

指导教师： 曹琼、刘小洋

时 间： 2026 年 5 月

评分依据	成绩
评分点 1: 能够对给定的简单语言设计其上下文无关文法和属性文法，对该语言的编译器进行分析，确定所设计的编译器的功能和应用环境，设计可行的设计方案。	
评分点 2: 能够选择合适的测试工具，设计多组测试数据测试所实现编译器的功能，并能正确评价所选工具和所实现的编译器的局限性	

目录

目录.....	2
1、课程设计目的.....	3
2. 课程设计内容.....	3
3. 方案设计（重点描述）.....	3
3.1 项目整体结构说明.....	3
3.2 题目一：整合 Sample 语言编译程序及可视化.....	5
3.3 题目二：任务 3.1 生成自选目标机的汇编代码.....	6
3.4 题目三：任务 3.2 实现中间代码解释器.....	7
3.5 题目四：任务 4.1 基于自动机的日志关键词扫描器.....	9
3.6 题目五：任务 4.2 四元式到 LLVM IR 的简化转换器.....	10
3.7 题目六：任务 4.3 基于正则表达式和语法分析的编程 IDE.....	11
3.8 题目七：任务 4.4 编译前端局部优化与流图分析系统.....	12
4. 测试及分析（重点描述）.....	13
4.1 测试用例来源.....	13
4.2 测试方法.....	13
4.3 测试用例设计.....	14
4.4 典型测试分析.....	14
4.4.1 词法分析测试.....	14
4.4.2 语法分析测试.....	15
4.4.3 语义分析测试.....	15
4.4.4 中间代码与解释执行测试.....	15
4.4.5 MASM16 汇编生成测试.....	15
4.4.6 日志正则与 NFA/DFA 测试.....	16
4.4.7 LLVM IR 生成测试.....	16
4.4.8 CFG 与 DAG 优化测试.....	16
4.4.9 GUI 编辑器功能测试.....	16
4.5 测试结论.....	17
5. 特色与拓展.....	17
（1）特色亮点.....	17
（2）拓展：从教学编译流程到工业级编译器.....	18
6. 总结及心得体会.....	19
6.1 课程设计总结.....	19
6.2 运行环境.....	19
6.3 程序限制条件.....	20
6.4 心得体会.....	20
附录 1：附录文件说明.....	21
1. 编译器核心代码目录说明.....	21
2. 测试用例目录说明.....	22
3. 输出文件目录说明.....	23
参考文献.....	24

1、课程设计目的

(1) 利用离散数学和形式语言的基本知识，对给定的简单语言设计其上下文无关文法和属性文法，对该语言的编译器进行分析，确定所设计的编译器的功能和应用环境，设计可行的设计方案。

(2) 选择合适的开发工具实现编译器的功能，并进行验证。

(3) 选择合适的测试工具，设计多组测试数据测试所实现编译器的功能，评价所选工具和所实现的编译器的局限性。

2. 课程设计内容

任务 3.1：生成自选目标机的汇编代码

将编译过程中生成的中间代码进一步转换为目标机汇编代码。本项目中需要根据四元式或目标代码结构生成可阅读的汇编形式，并使其能够体现变量赋值、算术运算、条件跳转、函数调用和返回等基本程序结构。

任务 3.2：实现中间代码解释器

对编译器生成的中间代码进行解释执行。该任务需要读取四元式等中间代码，模拟程序运行过程，处理变量赋值、表达式计算、条件判断、循环跳转、函数调用等操作，并输出程序的解释执行结果。

任务 4.1：基于自动机的日志文件关键词扫描器

利用词法分析和自动机构造思想，对日志文件中的关键信息进行识别和提取。该任务需要使用正则表达式描述日期、时间、日志级别、IP 地址、用户、状态码等关键词规则，并展示对应的 NFA、DFA 构造结果，最终完成日志关键信息扫描与输出。

任务 4.2：四元式到 LLVM IR 的简化转换器

将编译器生成的四元式转换为 LLVM IR 风格的中间代码。该任务需要识别普通变量、常量和临时变量，支持赋值、加减乘除、返回语句、比较语句和条件跳转等操作，并生成类似 `alloca`、`load`、`store`、`add`、`sub`、`mul`、`sdiv`、`icmp`、`br`、`ret` 等 LLVM IR 指令。

任务 4.3：基于正则表达式和语法分析的编程 IDE

实现一个面向类 C 语言程序的简单编程环境。该任务需要在编辑器中对代码进行实时扫描，完成关键字和函数名高亮、代码自动缩进，并对词法、语法、语义错误进行实时提示，从而提升程序编写和调试的交互体验。

任务 4.4：编译前端局部优化与流图分析系统

对中间代码进行基本块划分、控制流图构建和局部优化。该任务需要识别基本块入口语句，构造 CFG 控制流图，并在基本块内部使用 DAG 技术识别公共子表达式、合并重复计算，最终生成优化后的中间代码，并展示优化前后的对比结果。

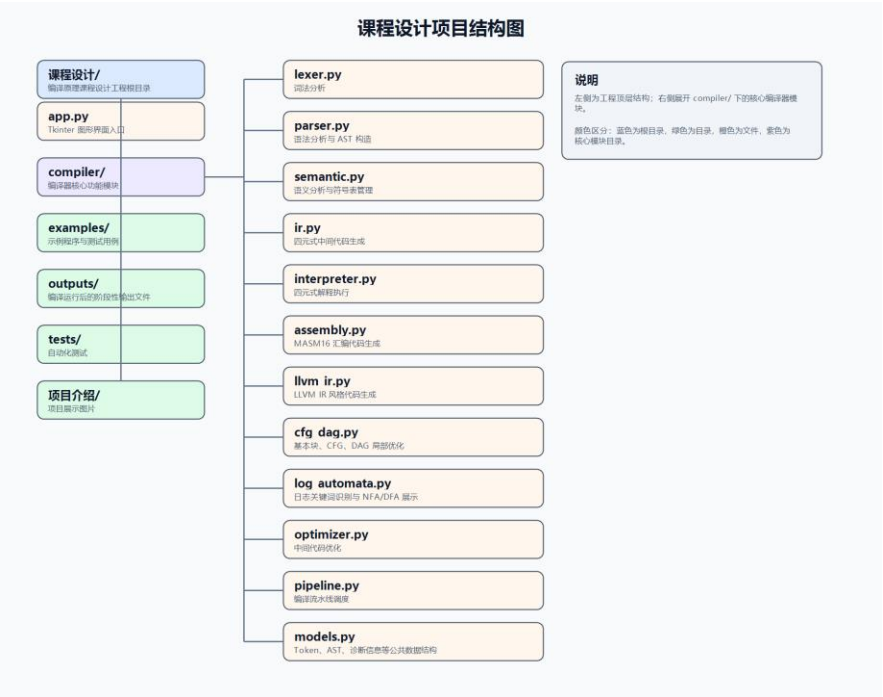
3. 方案设计（重点描述）

3.1 项目整体结构说明

本项目是一个面向类 C 语言的编译原理课程设计系统，整体采用“图形化界面 + 编译器核心模块 + 测试用例 + 输出文件”的结构进行组织。系统不仅实现了词法分析、语法分析、语义分析和中间代码生成等基本编译流程，还进一步扩展了中间代码解释执行、LLVM

IR 生成、MASM16 汇编代码生成、日志文件关键词识别、控制流图分析和 DAG 局部优化等功能。

项目目录结构如下图所示：



图表 1 课程设计结构图

其中，app.py 是整个系统的图形化界面入口，负责构建 Tkinter 桌面程序，实现源码编辑、文件打开与保存、编译运行、结果展示、日志识别和导出等功能。用户可以在左侧编辑区输入类 C 语言源程序，点击“运行”后，系统会自动调用完整编译流程，并在右侧结果区展示各阶段结果。

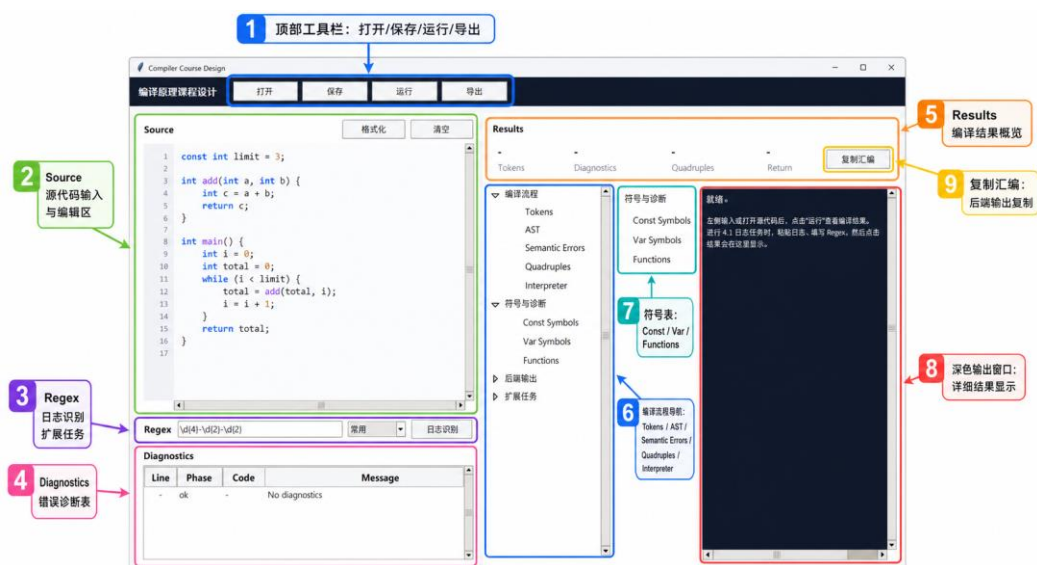
compiler/ 目录是项目的核心部分，包含各个编译阶段和扩展任务的实现模块。例如，lexer.py 负责词法分析，parser.py 负责语法分析和 AST 构造，semantic.py 负责语义检查和符号表管理，ir.py 负责生成四元式中间代码，interpreter.py 负责解释执行中间代码，assembly.py 负责生成 MASM16 汇编代码，llvm_ir.py 负责生成 LLVM IR 风格代码，cfg_dag.py 负责基本块划分、控制流图构建和 DAG 局部优化，log_automata.py 负责日志关键词识别以及 NFA、DFA 展示。

examples/ 目录中存放了用于演示和测试的源程序，包括正常编译程序、语义错误程序、优化测试程序、日志识别样例以及系统测试用例。通过这些测试文件，可以验证编译器各阶段功能是否正确。

outputs/ 目录用于保存系统运行后生成的各阶段输出文件，例如词法分析结果 tokens.txt、语法树 ast.txt、语义错误 semantic_errors.txt、四元式 quads.txt、解释执行结果 interpreter.txt、LLVM IR 输出 llvm_ir.txt、汇编代码 assembly.asm、基本块和控制流图分析结果等。这些文件便于后续查看、调试和撰写实验报告。

tests/ 目录用于存放自动化测试代码，主要用于验证编译流水线的正确性，保证词法分析、语法分析、语义分析、中间代码生成和扩展任务能够稳定运行。

系统整体功能结构如下图所示：



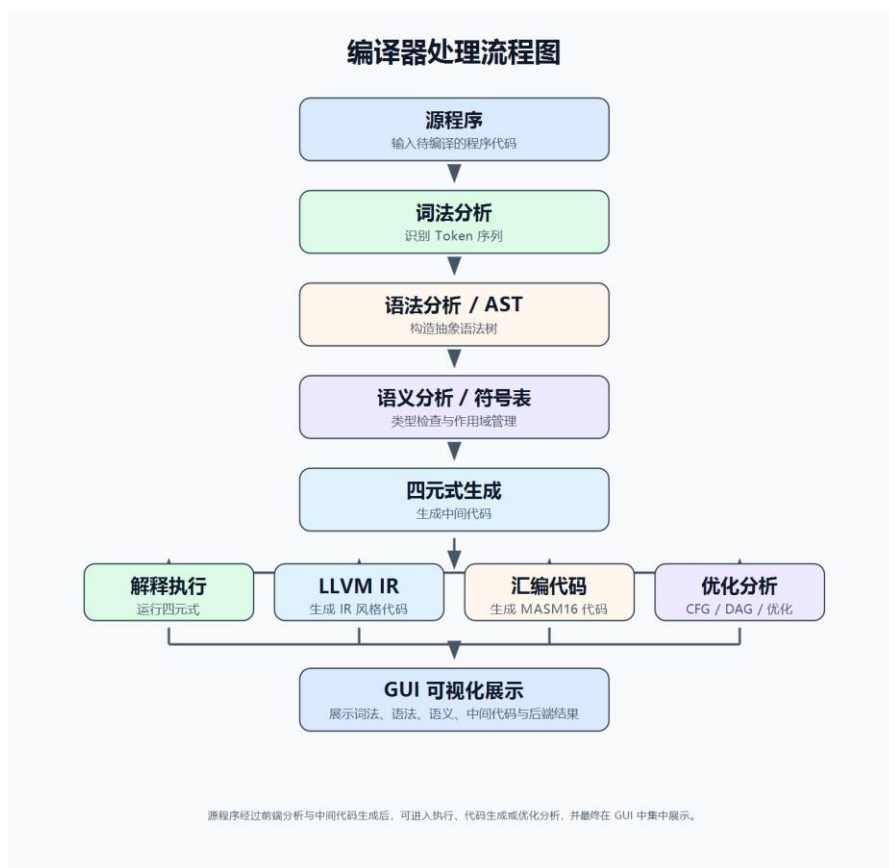
图表 2 项目成品介绍图

从整体流程来看，本项目以源程序作为输入，首先经过词法分析生成 Token 序列，再由语法分析器构造 AST 抽象语法树。随后，语义分析模块根据 AST 进行符号表构建和语义错误检查。在语义分析通过后，系统继续生成四元式中间代码，并在此基础上完成解释执行、LLVM IR 转换、汇编代码生成、基本块划分、CFG 构建和 DAG 优化等后续任务。

这种结构的优点是模块划分清晰，各阶段之间通过统一的数据结构传递结果，既便于单独调试某一编译阶段，也便于在 GUI 中集中展示完整编译过程。通过可视化界面，用户能够直观看到源程序从输入到词法分析、语法分析、语义分析、中间代码生成以及后端输出的完整过程，从而更好地理解编译器各组成部分之间的关系。

3.2 题目一：整合 Sample 语言编译程序及可视化

本项目首先对 Sample 类 C 语言编译程序进行了整体整合与可视化设计。系统以源代码作为输入，依次完成词法分析、语法分析、语义分析、中间代码生成、解释执行、目标代码生成和优化分析等步骤，并通过图形化界面集中展示各阶段结果。



图表 3 编译流水线结构图

在实现上, 项目通过 `pipeline.py` 统一调度各个编译阶段。用户在 GUI 左侧编辑区输入源程序后, 点击“运行”按钮, 系统会先调用词法分析器生成 Token 序列, 再调用语法分析器构造 AST 抽象语法树, 随后进行语义分析和符号表构建。如果程序能够继续处理, 则生成四元式中间代码, 并进一步完成解释执行、LLVM IR 生成、汇编代码生成、基本块划分、CFG 构建和 DAG 优化等操作。

语法分析阶段采用递归下降分析思想, 根据函数定义、变量声明、表达式、条件语句、循环语句和返回语句等语法结构逐步构造 AST。AST 中每个节点表示一种语法成分, 例如 `Program`、`FunctionDef`、`VarDecl`、`IfStmt`、`WhileStmt`、`ReturnStmt` 等。通过树形结构, 可以清晰表达源程序中语句之间的层次关系, 为后续语义分析和中间代码生成提供基础。

语义分析阶段主要完成符号表管理和错误检查。系统使用符号表栈维护不同作用域中的变量、常量和函数信息。当进入函数体或复合语句时创建新的作用域, 退出时弹出作用域。查找变量时从当前作用域向外层作用域逐层查找, 从而支持局部变量覆盖和嵌套作用域。系统能够检查重复定义、未声明变量、函数参数不匹配、返回类型错误、非法 `break` 或 `continue` 等问题。

可视化方面, 系统使用 Tkinter 构建桌面界面。左侧为源码编辑器, 支持行号显示、关键字高亮、函数名高亮、自动缩进和实时错误提示; 右侧为结果展示区, 可以查看 Tokens、AST、语义错误、符号表、四元式、解释执行结果、LLVM IR、汇编代码、CFG 和 DAG 等内容。通过这种设计, 用户能够直观看到源程序从输入到各编译阶段输出的完整变化过程。

3.3 题目二：任务 3.1 生成自选目标机的汇编代码

本项目完成了任务书中 3.1 的要求, 即根据中间代码生成自选目标机的汇编代码。项

目选择 MASM16 风格汇编作为目标代码形式，由 assembly.py 模块负责将四元式翻译为汇编指令，并将结果输出到 outputs/assembly.asm 文件中。



图表 4 汇编代码生成流程图

汇编代码生成的输入是前端产生的四元式。四元式是一种结构统一的中间表示，通常形式为 (op, arg1, arg2, result)。系统根据 op 的不同类型选择不同的汇编翻译规则。例如，赋值四元式 = 会被翻译为数据传送指令；加减乘除四元式会被翻译为加载操作数、执行算术运算、保存结果的指令序列；条件跳转四元式会被翻译为 CMP 比较指令和条件跳转指令。

在变量和内存管理方面，系统采用栈帧思想分配局部变量空间。函数开始时保存 BP，并根据局部变量数量调整 SP，为局部变量分配栈空间。局部变量按照顺序分配相对于 BP 的偏移地址，函数参数则通过参数偏移进行访问。这样可以较好地模拟真实汇编程序中的函数调用和局部变量管理方式。

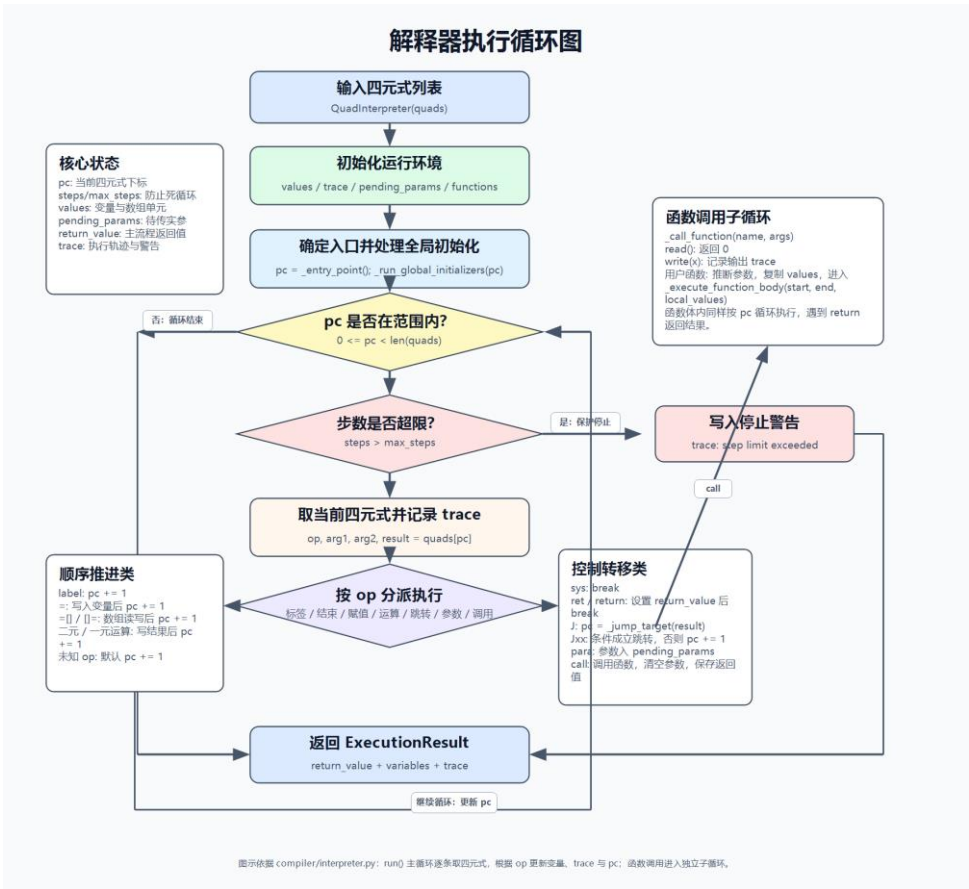
函数调用是汇编生成中的重点。系统在四元式层面使用 para 表示参数准备，使用 call 表示函数调用。翻译到汇编时，需要将参数压栈或放入约定位置，调用目标函数，并在函数返回后恢复栈空间。返回值主要通过寄存器保存，再写入对应变量的临时变量中。

本部分实现的难点在于四元式与汇编指令之间存在抽象层次差异。四元式不关心寄存器、栈空间和变量地址，而汇编代码必须明确处理这些底层细节。因此，在实现过程中需要设计变量地址分配规则、跳转标签生成规则、函数调用规则和返回值保存规则，保证生成的汇编代码逻辑与原程序一致。

3.4 题目三：任务 3.2 实现中间代码解释器

本项目完成了任务书中 3.2 的要求，实现了一个基于四元式的中间代码解释器。解释

器由 interpreter.py 模块实现，用于直接模拟四元式的执行过程，并输出程序返回值、变量最终状态和执行轨迹。



图表 5 解释器执行循环图

解释器的核心思想是使用程序计数器 `pc` 按顺序读取四元式，并根据当前四元式的操作类型执行对应动作。对于赋值四元式，解释器会计算右侧值并更新变量表；对于算术四元式，解释器会读取两个操作数，完成加、减、乘、除、取模等运算，并将结果保存到临时变量或目标变量中；对于条件跳转四元式，解释器会根据比较结果决定是否修改 `pc`；对于无条件跳转四元式，则直接跳转到目标四元式编号。

在数据存储方面，解释器使用字典结构维护变量和临时变量的值。普通变量、临时变量和数组元素都可以通过名称进行访问。数组元素使用类似 `a[0]` 的形式保存，从而在不引入真实内存模型的情况下模拟数组读写。

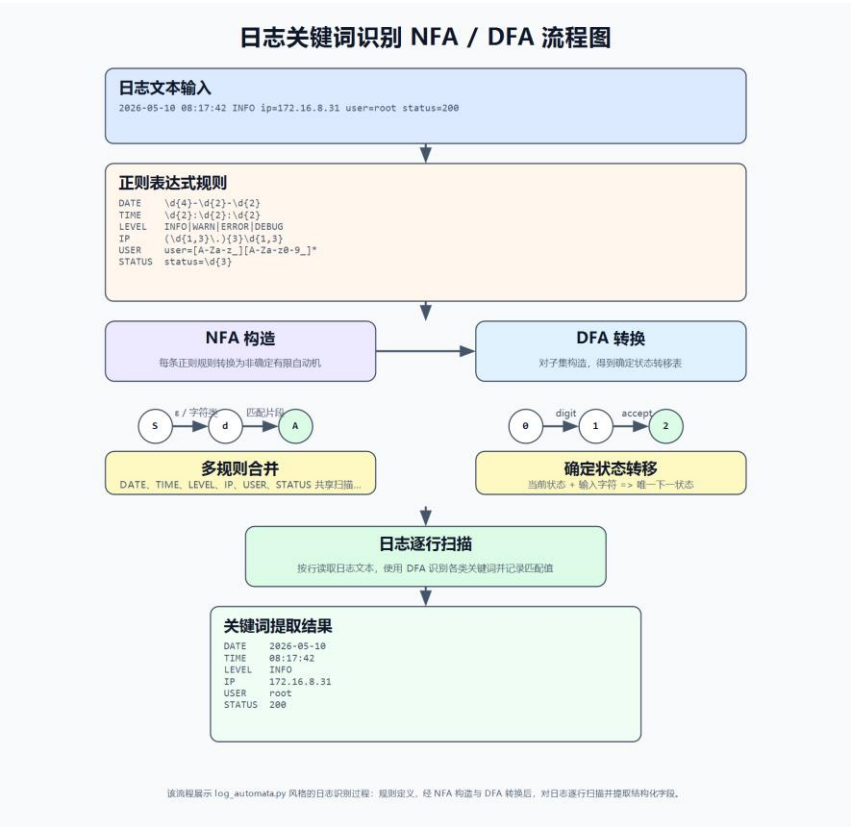
函数调用也是解释器的重要部分。系统使用 `para` 四元式收集实参，遇到 `call` 四元式时进入对应函数执行。函数执行时会建立局部变量环境，将实参绑定到形参，再从函数入口开始执行四元式，直到遇到 `return` 或 `ret`。此外，系统还内置了 `read` 和 `write` 函数，其中 `read` 用于模拟输入，`write` 用于模拟输出，方便测试程序的输入输出行为。

解释器还设计了执行轨迹输出功能。每执行一条四元式，系统都会记录当前执行的编号、四元式内容和对应含义。这样不仅可以得到最终结果，还可以观察程序是如何一步步运行的，便于调试循环、条件分支和函数调用。

本部分的主要难点在于正确维护程序执行顺序。尤其是在循环、条件跳转和函数调用场景中，`pc` 的变化不再是简单递增。为了避免错误程序导致无限循环，解释器还设置了最大执行步数限制，当执行步数超过限制时会停止运行并给出运行时警告。

3.5 题目四：任务 4.1 基于自动机的日志关键词扫描器

本项目完成了任务书中 4.1 的要求，实现了基于自动机思想的日志文件关键词扫描器。该功能由 log_automata.py 模块实现，主要用于从日志文件中提取日期、时间、日志级别、IP 地址、状态码、用户和动作等关键信息。



图表 6 日志关键词识别流程图

系统首先使用正则表达式描述不同日志字段的构成规则。例如，日期字段使用 `\d{4}-\d{2}-\d{2}` 描述，时间字段使用 `\d{2}:\d{2}:\d{2}` 描述，IP 地址使用数字和点号组合规则描述，日志级别则匹配 INFO、WARN、ERROR、DEBUG 等关键字。

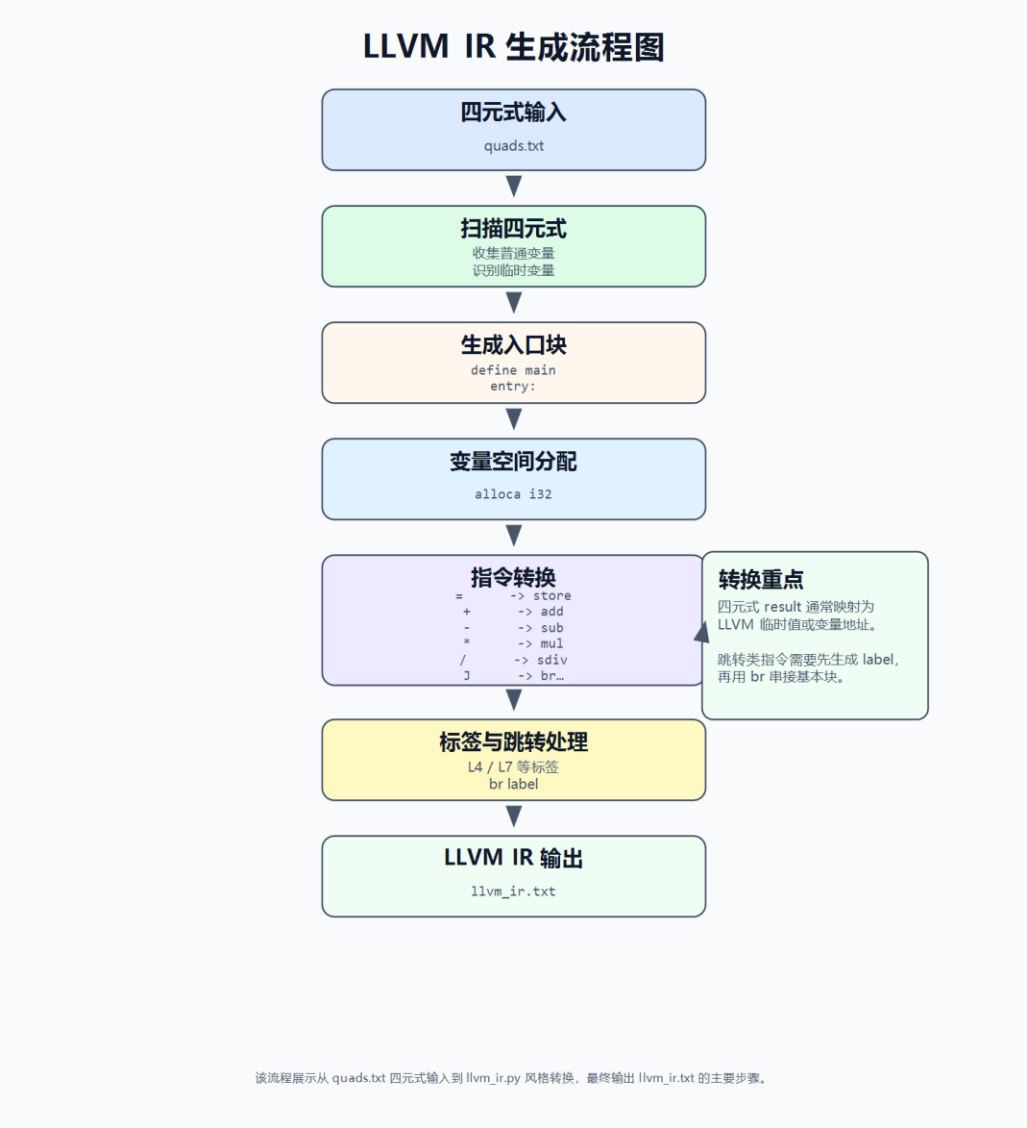
在自动机构造展示方面，系统根据正则表达式片段生成对应的 NFA 和 DFA 文本描述，并展示状态集合、开始状态、接受状态和状态转移关系。用户也可以在 GUI 中输入自定义正则表达式，系统会根据该表达式生成对应的 NFA、DFA 和 DFA 转换表。

日志扫描时，系统逐行读取日志内容，使用规则集合对每一行进行匹配。匹配结果包括字段类型、字段值、所在行号和列范围，并在 GUI 的 Log Extract 面板中展示，同时写入 `outputs/log_extract.txt` 文件。NFA 和 DFA 的构造结果分别写入 `outputs/log_nfa.txt` 和 `outputs/log_dfa.txt`。

本部分的难点在于日志格式并不完全统一。不同日志中同一字段可能有不同写法，例如状态码既可能以 `status=200` 形式出现，也可能直接作为三位数字出现。因此，正则规则需要兼顾覆盖范围和误匹配控制。为了避免同一位置被多个规则重复识别，系统在扫描时还对匹配区间进行了重叠判断。

3.6 题目五：任务 4.2 四元式到 LLVM IR 的简化转换器

本项目完成了任务书中 4.2 的要求，实现了四元式到 LLVM IR 风格代码的转换。该功能由 `llvm_ir.py` 模块实现，目标是将课程中使用的四元式中间代码转换为更接近工业编译器中间表示的 LLVM IR 形式。



图表 7 LLVM_IR 生成流程图

系统首先扫描四元式，收集普通变量，并在 LLVM IR 的入口基本块中为这些变量生成 `alloca` 指令。普通变量在 LLVM IR 中被视为内存地址，因此参与运算前需要通过 `load` 指令取值，赋值时需要通过 `store` 指令写回。临时变量则以 SSA 风格的 `%t1`、`%t2` 等形式直接保存计算结果。

对于算术四元式，系统将 `+`、`-`、`*`、`/`、`%` 分别转换为 `add`、`sub`、`mul`、`sdiv`、`srem` 指令。对于比较四元式，系统生成 `icmp` 指令，并根据比较类型选择 `sgt`、`slt`、`eq`、`ne` 等谓词。对于条件跳转四元式，系统生成 `br il` 条件分支指令；对于无条件跳转，则生成 `br label` 指令；对于返回语句，则生成 `ret i32` 指令。

在标签处理方面，系统根据跳转目标四元式编号生成 LLVM 基本块标签，例如 `L4`、`L7` 等。每个基本块需要以终结指令结束，因此系统在生成标签前会检查上一条指令是否已

经是 `br` 或 `ret`，必要时补充跳转指令，保证生成的 IR 结构合法。

本部分的难点在于四元式和 LLVM IR 的变量模型不同。四元式可以直接使用变量名参与运算，而 LLVM IR 中变量地址和值需要严格区分。因此，系统在生成每条运算指令前，需要判断操作数是常量、临时变量还是普通变量，并选择直接使用或先 `load`。

3.7 题目六：任务 4.3 基于正则表达式和语法分析的编程 IDE

本项目完成了任务书中 4.3 的要求，实现了一个面向类 C 语言的简单编程 IDE。该功能主要由 `app.py` 和 `source_format.py` 实现，结合词法分析、语法分析和语义分析模块，为用户提供代码编辑、格式化、高亮和实时错误提示功能。

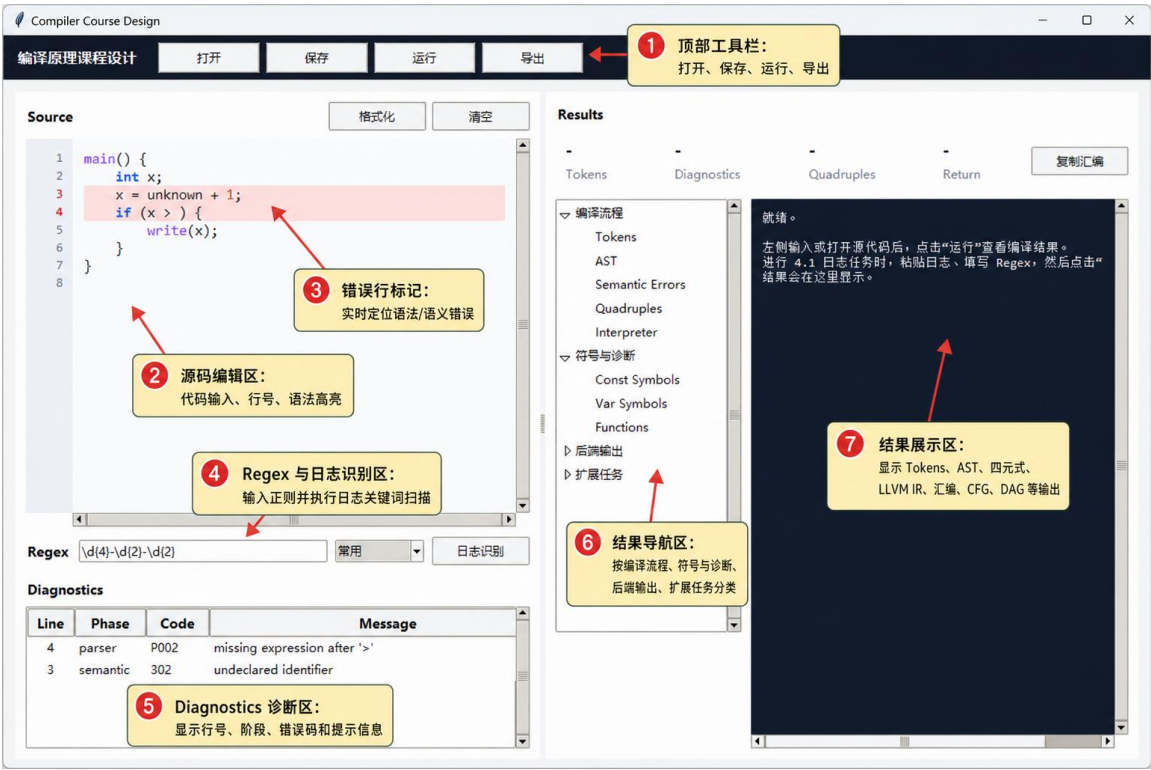


图 8 GUI 界面功能标注图

界面左侧为源码编辑区，支持行号显示、关键字高亮、函数名高亮、数字高亮、字符串高亮和注释高亮。系统通过词法扫描和正则匹配识别不同类型的代码元素，并在文本框中设置不同样式，使程序结构更加清晰。

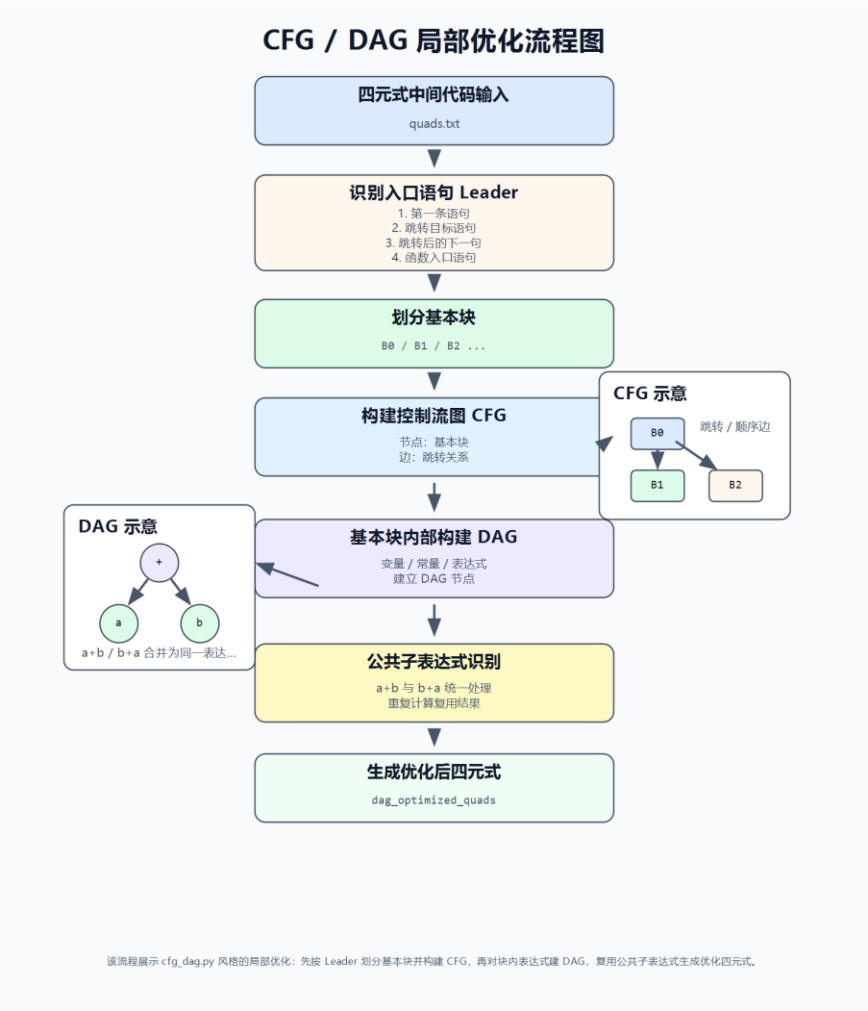
代码格式化由 `source_format.py` 实现，主要依据花括号层级调整缩进。当遇到 `{` 时增加缩进层级，遇到 `}` 时减少缩进层级，从而自动整理代码结构。用户可以点击“格式化”按钮对当前代码进行统一排版。

实时错误提示是 IDE 的核心功能之一。当用户修改代码后，系统会延迟触发诊断流程，调用词法分析、语法分析和语义分析模块获取错误信息。错误信息会显示在下方 **Diagnostics** 面板中，包括错误所在行、错误阶段、错误码和错误描述。同时，出错行会在编辑区中以醒目方式标记，用户点击错误项可以快速定位到对应代码位置。

本部分的难点在于实时分析需要在准确性和响应速度之间取得平衡。如果每输入一个字符就立即完整编译，会导致界面卡顿；如果延迟过长，又会降低交互体验。因此，系统采用延迟调度方式，在用户停止输入一小段时间后再进行诊断，从而提高界面流畅度。

3.8 题目七：任务 4.4 编译前端局部优化与流图分析系统

本项目完成了任务书中 4.4 的要求，实现了基于四元式的基本块划分、控制流图构建和 DAG 局部优化功能。该功能由 `cfg_dag.py` 模块实现，并在 GUI 中提供 Basic Blocks、CFG、DAG 和 DAG Optimized Quads 等结果展示。



图表 9 CFG_DAG 优化流程图

系统首先对线性四元式序列进行基本块划分。基本块入口语句包括程序入口、函数入口、跳转目标语句以及跳转语句的下一条语句。确定入口语句后，系统按照入口位置将四元式序列切分为多个基本块，每个基本块内部按顺序执行，中间没有额外跳入或跳出。

在基本块划分基础上，系统进一步构建控制流图 CFG。CFG 中每个节点表示一个基本块，边表示基本块之间可能发生的控制转移。如果某个基本块以无条件跳转结束，则只连接跳转目标；如果以条件跳转结束，则同时连接条件成立目标和顺序后继基本块；如果不是跳转或终止语句，则连接下一个基本块。

局部优化阶段使用 DAG 技术处理每个基本块内部的算术表达式。系统为变量、常量和表达式建立 DAG 节点，并通过表达式键判断是否已经存在相同计算。如果发现公共子表达式，则不再重复生成计算四元式，而是直接复用已有结果。例如，当一个基本块中多次出现相同的 $a + b$ 时，后续结果可以通过赋值复用前一次计算结果。

对于加法和乘法这类满足交换律的运算，系统会对两个操作数进行统一排序，使 $a + b$ 和 $b + a$ 能够被识别为同一个表达式。优化完成后，系统重新生成优化后的四元式，并展

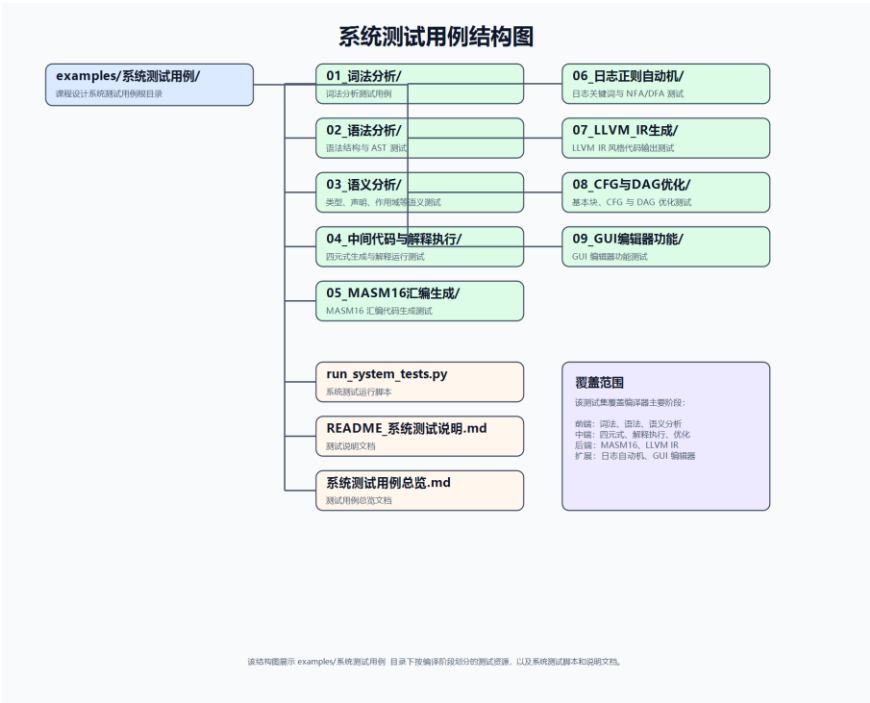
示优化前后的对比。

本部分的难点在于优化必须保证语义不变。DAG 优化只能在基本块内部进行，不能跨越控制流边界随意合并表达式。同时，在处理赋值和临时变量时，需要维护变量当前对应的值，避免错误复用已经失效的表达式结果。

4. 测试及分析（重点描述）

4.1 测试用例来源

本项目测试用例主要来自 examples/系统测试用例 目录。该目录按照功能模块划分测试用例，覆盖词法分析、语法分析、语义分析、中间代码与解释执行、MASM16 汇编生成、日志正则与 NFA/DFA、LLVM IR 生成、CFG 与 DAG 优化、GUI 编辑器功能等内容。测试用例目录结构如下：



图表 10 系统测试用例结构图

4.2 测试方法

本项目采用自动化测试和 GUI 手动测试相结合的方式进行验证。

自动化测试在项目根目录运行：

python examples/系统测试用例/run_system_tests.py

预期输出为：

system tests: PASS covered: lexical, syntax, semantic, IR/interpreter, MASM16, log regex NFA/DFA, LLVM IR, CFG/DAG, GUI editor inputs

GUI 手动测试时，依次打开 examples/系统测试用例 中的测试文件，点击“运行”或“日志识别”，查看右侧结果区中的 Tokens、AST、Semantic Errors、Quadruples、Interpreter、LLVM IR、Assembly、Basic Blocks、CFG、DAG 等输出，并与预期结果进行对照。

4.3 测试用例设计

系统测试用例表				
测试模块	测试文件	测试类型	主要测试功能	预期结果
词法分析	lexical_all_tokens.c	正常用例	关键字、标识符、整数、字符、字符串、运算符	无诊断, Tokens 正常生成
词法分析	lexical_errors.c	错误用例	非法字符、空字符常量、未闭合字符串	产生词法错误 L003、L004
语法分析	syntax_control_flow.c	正常用例	函数定义、if/else、for、return	AST 中包含控制流节点
语法分析	syntax_errors.c	错误用例	缺少分号、表达式缺失、括号错误	产生 P001、P002
语义分析	semantic_symbols_ok.c	正常用例	常量表、变量表、函数表	Functions 中包含函数信息
语义分析	semantic_errors.c	错误用例	重复声明、未声明变量、参数错误、常量赋值	产生 S01、S02、S05、S09
中间代码与解释执行	interpreter_loop_factorial.c	正常用例	循环、乘法、解释执行	输出 builtin write(120)
中间代码与解释执行	interpreter_branch_function.c	正常用例	函数调用、条件分支	输出 builtin write(7)
中间代码与解释执行	interpreter_runtime_warning.c	边界用例	除零、取模等保护	不崩溃, 显示运行时警告
MASM16 汇编生成	assembly_basic_masm16.c	正常用例	段式汇编、循环、函数调用	输出 data segment、code segment、main、add
MASM16 汇编生成	assembly_read_write.c	正常用例	DOS 输入输出过程	输出 CALL read、CALL write、read proc near
MASM16 汇编生成	assembly_recursive_factor.c	正常用例	递归函数、栈帧参数	输出 factor、CALL factor、ss[bp+4]
日志正则自动机	log_sample.log	正常用例	DATE、TIME、IP、STATUS、USER、ACTION	能够匹配日志关键字
日志正则自动机	log_mixed_formats.log	正常用例	混合格式日志、状态码	正则能够匹配多种格式
日志正则自动机	no_match.log	边界用例	无匹配日志	显示无匹配提示
LLVM IR 生成	llvm_branch_call.c	正常用例	函数调用、分支、LLVM 验证	包含 define i32 @main、br i1、ret i32
LLVM IR 生成	llvm_module_loop.c	正常用例	循环、取模、条件跳转	包含 srem i32、br i1
CFG 与 DAG 优化	cfg_if_else.c	正常用例	Leaders、基本块划分、if/else CFG	Basic Blocks 和 CFG 输出正确
CFG 与 DAG 优化	dag_common_subexpr.c	正常用例	公共子表达式识别	DAG 中出现 commonc
CFG 与 DAG 优化	cfg_dag_complex.c	综合用例	多基本块 CFG、多块 DAG、优化前后对比	优化后目标代码行数减少
GUI 编辑器功能	gui_realtime_errors.c	错误用例	实时错误标记、诊断表	产生 P002、S02
GUI 编辑器功能	gui_format_highlight.c	正常用例	格式化、自动缩进、高亮	格式化的语法高亮

该表按测试模块汇总测试文件、测试类型、主要功能与预期结果。覆盖词法、语法、语义、中间代码、后端生成、日志自动机、优化与 GUI。

图表 11 系统测试用例表

4.4 典型测试分析

4.4.1 词法分析测试

词法分析测试用例位于 examples/ 系统测试用例 /01_词法分析。其中，lexical_all_tokens.c 用于测试系统对关键字、标识符、整数、字符、字符串、算术运算符、关系运算符、逻辑运算符和分隔符的识别能力。运行后，系统能够在 Tokens 面板中输出对应的 Token 文本、种别码和所在行号，且无诊断错误。

lexical_errors.c 用于测试词法错误处理能力，包含非法字符、空字符常量和未闭合字符串等情况。运行后，Diagnostics 面板能够显示对应的词法错误，如 L003、L004，说明词法分析器能够在遇到非法输入时给出明确诊断，而不是直接中断程序运行。

4.4.2 语法分析测试

语法分析测试用例位于 examples/ 系统测试用例 /02_ 语法分析。syntax_control_flow.c 和 syntax_nested_loops.c 分别用于测试函数定义、if/else、for、while、return 以及嵌套循环等语法结构。运行后,AST 输出中能够看到 ForStmt、WhileStmt、IfStmt、ReturnStmt 等节点,说明语法分析器能够正确构造抽象语法树。

syntax_errors.c 用于测试语法错误恢复能力。该文件中包含缺少分号、表达式缺失和括号结构错误等问题。运行后,系统能够产生 P001、P002 等语法诊断,并在 GUI 中标记错误位置,说明系统能够对常见语法错误进行定位和提示。

4.4.3 语义分析测试

语义分析测试用例位于 examples/ 系统测试用例 /03_ 语义分析。semantic_symbols_ok.c 用于测试正常程序的符号表构建,运行后 Const Symbols、Var Symbols 和 Functions 面板能够显示常量、变量和函数信息。例如 Functions 表中包含函数名、返回类型和参数类型。

semantic_errors.c 用于测试语义错误诊断能力,覆盖重复声明、未声明标识符、函数参数个数错误、常量赋值错误等情况。系统运行后能够产生 301、302、305、309 等错误码。测试结果说明语义分析模块能够正确维护符号表,并根据符号表信息检查程序中的语义问题。

4.4.4 中间代码与解释执行测试

中间代码与解释执行测试用例位于 examples/ 系统测试用例 /04_ 中间代码与解释执行。interpreter_loop_factorial.c 用于测试循环、乘法和解释执行功能。程序通过循环计算阶乘,运行后 Interpreter 面板中输出 builtin write(120),说明四元式生成和解释执行结果正确。

interpreter_branch_function.c 用于测试函数调用和条件分支。系统生成的四元式中包含 para、call、条件跳转等指令,解释器能够正确完成参数传递和函数返回,最终输出 builtin write(7)。

interpreter_runtime_warning.c 用于测试边界情况,包含除零或取模零等风险操作。运行后系统不会崩溃,而是在 Interpreter 输出中给出 runtime warning,说明解释器具有一定的运行时保护能力。

4.4.5 MASM16 汇编生成测试

MASM16 汇编测试用例位于 examples/ 系统测试用例 /05_ MASM16 汇编生成。assembly_basic_masm16.c 用于测试基本汇编结构、循环和函数调用。运行后 Assembly 输出中包含 assume cs:code,ds:data,ss:stack,es:extended、data segment、code segment、main: 和 add: 等内容,说明系统能够生成完整的 MASM16 汇编框架。

assembly_read_write.c 用于测试输入输出过程。生成的汇编代码中包含 CALL read、CALL write、read proc near、write proc near 等指令和过程定义,说明系统能够为内置输入输出函数生成对应的汇编支持。

assembly_recursive_factor.c 用于测试递归函数和栈传参。生成代码中可以看到 factor:、

CALL factor、ss:[bp+4] 等内容，说明系统通过栈帧访问函数参数，并支持递归调用场景。

4.4.6 日志正则与 NFA/DFA 测试

日志正则自动机测试用例位于 examples/系统测试用例/06_日志正则自动机。测试时将 log_sample.log 内容粘贴到左侧编辑器，在 Regex 输入框中输入 IP 正则：

```
(?:\d{1,3}\.){3}\d{1,3}
```

点击“日志识别”后，Log Extract 面板能够显示匹配到的 IP 地址及其所在行列位置。NFA Graph、DFA Graph 和 DFA Table 面板能够展示正则表达式对应的自动机构造过程。

log_mixed_formats.log 用于测试混合格式日志，验证系统对不同字段写法的适应能力；no_match.log 用于测试无匹配情况，系统能够显示无匹配提示而不会报错。测试结果说明日志关键词扫描器能够完成正则匹配、自动机展示和结果输出。

4.4.7 LLVM IR 生成测试

LLVM IR 测试用例位于 examples/系统测试用例/07_LLVM_IR 生成。llvm_branch_call.c 用于测试函数调用、分支结构和局部变量转换。运行后 LLVM IR 输出中包含 define i32 @main、alloca i32、load、store、br i1 和 ret i32 等指令，说明四元式能够被转换为 LLVM IR 风格代码。

llvm_modulo_loop.c 用于测试循环和取模操作。生成结果中包含 srem i32 和条件跳转指令，说明系统能够处理取模和循环控制结构。LLVM Verify 面板显示 Internal verifier: PASS，说明生成的 LLVM IR 在内部结构检查上通过验证。

4.4.8 CFG 与 DAG 优化测试

CFG 与 DAG 测试用例位于 examples/系统测试用例/08_CFG 与 DAG 优化。cfg_if_else.c 用于测试 if/else 结构下的基本块划分和控制流图构建。运行后，Basic Blocks 面板中能够看到 Leaders 表，CFG 面板中能够看到基本块之间的前驱和后继关系。

dag_common_subexpr.c 用于测试公共子表达式识别。程序中存在重复的 a + b 表达式，系统在 DAG 面板中输出 common: 信息，并在 DAG Optimized Quads 中减少重复计算。

cfg_dag_complex.c 用于综合测试多基本块 CFG、多块 DAG 和优化后目标代码生成。测试结果中，Optimized Target Code 行数少于 Target Code，说明 DAG 优化结果已经进入后续目标代码生成阶段，而不只是停留在分析展示层面。

4.4.9 GUI 编辑器功能测试

GUI 编辑器测试用例位于 examples/系统测试用例/09_GUI 编辑器功能。gui_realtime_errors.c 用于测试实时错误标记和下方诊断表。程序中包含表达式缺失和未声明变量，系统能够产生 P002 和 302 错误，并在源码编辑区标红对应行。

gui_format_highlight.c 用于测试格式化、自动缩进和代码高亮。点击“格式化”后，代码缩进变得清晰，关键字和函数名能够以不同颜色显示，说明 IDE 的基础编辑辅助功能正常。

4.5 测试结论

通过 examples/系统测试用例 中各类测试文件的验证，本项目完成了从编译前端到后端扩展、从自动机应用到 GUI 交互的系统级测试。测试结果表明，项目能够正确完成词法分析、语法分析、语义分析、四元式生成、中间代码解释执行、MASM16 汇编生成、LLVM IR 生成、日志关键词识别、CFG/DAG 优化和实时错误提示等功能。

正常用例能够顺利生成预期输出，错误用例能够给出明确诊断，边界用例能够保持系统稳定运行，说明本项目整体功能完整，符合课程设计任务要求。

5. 特色与拓展

(1) 特色亮点

本次课程设计的特色在于没有只停留在单个编译阶段的实现，而是将词法分析、语法分析、语义分析、中间代码生成、解释执行、LLVM IR 生成、MASM16 汇编生成、日志自动机识别、CFG/DAG 优化和图形化界面整合为一个完整系统。用户可以在 GUI 中输入源程序，一次运行后查看编译全过程的阶段性的结果，能够直观理解源代码从文本到中间表示、再到目标代码的转换过程。

首先，系统具有较完整的编译流水线。项目通过 pipeline.py 统一调度各阶段模块，依次完成 Token 生成、AST 构造、符号表管理、四元式生成、解释执行、LLVM IR 转换和汇编代码生成。各阶段输出均可以在界面中查看，也可以导出到 outputs/ 目录，便于调试和报告分析。

其次，系统实现了较强的可视化展示能力。GUI 不仅提供源码编辑区、行号、语法高亮、自动缩进和实时错误提示，还提供 Tokens、AST、Semantic Errors、Quadruples、Interpreter、LLVM IR、Assembly、CFG、DAG 等结果查看入口。相比命令行输出，这种方式更适合教学展示，能够帮助理解编译器各阶段之间的联系。

第三，项目不仅实现了编译器前端，还扩展到了后端和优化部分。例如，系统能够将四元式转换为 MASM16 汇编代码，展示变量、表达式、跳转和函数调用如何映射到底层指令；同时也能生成 LLVM IR 风格代码，使课堂中的中间代码与现代工业编译器的 IR 建立联系。

第四，项目实现了中间代码解释器。解释器通过程序计数器模拟四元式执行过程，支持赋值、算术运算、条件跳转、循环、函数调用和内置 read/write，并输出执行轨迹。该功能能够帮助验证中间代码语义是否正确，也能直观展示程序运行过程。

第五，项目结合自动机理论实现了日志关键词扫描器。系统使用正则表达式描述 DATE、TIME、LEVEL、IP、STATUS、USER、ACTION 等日志字段，并展示对应的 NFA、DFA 和匹配结果。这一部分将词法分析中的正则表达式和自动机构造方法应用到日志分析场景，体现了编译原理知识在实际文本处理任务中的应用价值。

第六，项目实现了 CFG 与 DAG 局部优化。系统能够对四元式进行基本块划分，构建控制流图，并在基本块内部使用 DAG 识别公共子表达式，生成优化后的四元式和目标代码。该功能体现了从“能编译”到“能优化”的进一步扩展。

（2）拓展：从教学编译流程到工业级编译器

通过拓展阅读毕昇编译器相关资料,可以看到课堂中学习的编译流程与真实工业级编译器之间具有直接联系。根据 openEuler 文档介绍,毕昇编译器是华为编译器实验室面向鲲鹏等通用处理器架构打造的编译器工具链,强调高性能、高可信和易扩展,支持 C、C++、Fortran 等语言,并已融入 openEuler 生态。华为相关文档也说明,毕昇编译器基于 LLVM 开发,并针对鲲鹏平台进行了优化和改进。

毕昇编译器的定位不是教学实验中的小型编译器,而是面向真实服务器、操作系统和高性能计算场景的工业级工具链。它需要处理真实工程中的大规模代码,支持多语言、多平台、优化选项、链接、运行库和硬件相关优化。因此,它的复杂度远高于课程设计项目,但其基本思想仍然与课堂编译流程一致。

课堂编译流程与毕昇编译器之间的对应关系如下:

表格 1 课堂编译流程与毕昇编译器之间的对应关系

课堂编译阶段	本项目中的实现	毕昇编译器中的对应
词法分析	lexer.py 将源码切分为 Token	Clang 前端对 C/C++/Fortran 源码进行词法处理
语法分析	parser.py 构造 AST	前端根据语言语法构造语法树/语义结构
语义分析	semantic.py 检查符号表、类型、函数调用	前端进行类型检查、作用域分析、函数声明与定义检查
中间代码	ir.py 生成四元式,llvm_ir.py 生成 LLVM IR 风格代码	LLVM 使用 LLVM IR 作为核心中间表示
中间代码优化	optimizer.py、cfg_dag.py 进行常量折叠、DAG 优化、CFG 分析	LLVM 中端执行大量优化 Pass,如控制流优化、循环优化、向量化等
目标代码生成	assembly.py 生成 MASM16 汇编	LLVM 后端根据目标架构生成机器代码或汇编代码
运行与工具链	GUI 展示、导出输出文件	工业工具链还包括汇编器、链接器、运行库、调试信息和性能调优工具

通过对比可以发现,课堂中的词法分析、语法分析、语义分析、中间代码、优化和目标代码生成并不是孤立的理论知识,而是真实编译器的基础框架。区别在于,课程设计通常处理简化语言和小规模程序,而毕昇编译器需要处理完整的 C/C++/Fortran 语言规范、复杂硬件平台和真实工程性能要求。

在本项目中,我们实现的 LLVM IR 转换部分尤其体现了这种联系。课堂中使用四元式作为中间代码,便于理解表达式、跳转和函数调用;而工业编译器中通常使用更规范、更适合优化的 IR。LLVM IR 既能表达程序语义,又便于进行跨平台优化和后端代码生成。通过将四元式转换为 LLVM IR 风格代码,可以更直观地理解教学中间代码与工业级 IR 之间的关系。

毕昇编译器还体现了国产基础软件的重要性。编译器位于软件工具链的核心位置,上层应用、操作系统、数据库、高性能计算程序都需要依赖编译器生成高效可靠的机器代码。面向鲲鹏等国产硬件平台进行编译优化,可以更充分发挥硬件性能,提升国产软硬件生态的自

主能力。

通过本次拓展阅读，我认识到学习编译原理不仅是为了掌握词法分析、语法分析等课程知识，更重要的是理解现代软件工具链的底层机制。编译器连接了高级语言和硬件平台，既涉及语言设计，也涉及程序优化、体系结构和操作系统。掌握编译原理后，能够更好地理解真实开发工具的工作方式，也能更深入地理解国产基础软件在操作系统和处理器生态中的作用。

6. 总结及心得体会

6.1 课程设计总结

本次课程设计围绕类 C 语言编译系统展开，完成了从源程序输入到多种编译结果输出的完整流程。项目实现了词法分析、语法分析、语义分析、中间代码生成、中间代码解释执行、MASM16 汇编代码生成、LLVM IR 风格代码生成、日志关键词自动机识别、CFG 控制流分析、DAG 局部优化以及 GUI 可视化展示等功能。对应任务书中，我们完成了 3.1、3.2、4.1、4.2、4.3、4.4 等任务。

通过本次设计，我进一步理解了编译器各阶段之间的关系。词法分析将源程序转换为 Token 序列，语法分析根据文法规则构造 AST 抽象语法树，语义分析通过符号表检查变量、常量、函数和类型的合法性，中间代码生成将高级语言结构转换为四元式，后续再基于四元式完成解释执行、汇编生成、LLVM IR 转换和代码优化。整个过程体现了从文本、结构、语义到目标代码的逐层抽象和形式化转换。

在技术方面，本项目使用 Python 作为主要开发语言，使用 Tkinter 实现桌面 GUI，使用模块化结构组织编译器各阶段功能。开发过程中掌握了递归下降语法分析、符号表栈管理、四元式生成、程序计数器解释执行、函数调用模拟、正则表达式匹配、NFA/DFA 展示、基本块划分、控制流图构建和 DAG 公共子表达式优化等技术。

6.2 运行环境

本项目运行环境如下：

表格 2 运行环境

项目	配置
操作系统	Windows 10 Home China, 64 位
处理器	13th Gen Intel(R) Core(TM) i5-13500H
内存	约 16 GB
开发语言	Python
Python 版本	Python 3.12.13
图形界面库	Tkinter
运行终端	Windows PowerShell
开发与调试工具	VS Code / PowerShell / Python unittest

项目	配置
项目运行入口	app.py

启动方式为在项目根目录执行：

```
python app.py
```

6.3 程序限制条件

由于本项目是面向课程设计的教学型编译系统，并非完整工业级 C 编译器，因此仍存在一定限制：

支持的是类 C 语言子集，不支持完整 C 语言标准。

标识符、关键字、数字常量等词法单元不能跨行书写。

字符串字面量不支持跨行。

注释支持常见行注释和块注释，但不支持嵌套块注释。

语法分析主要覆盖变量声明、函数定义、表达式、赋值、条件语句、循环语句、函数调用和返回语句等常见结构。

类型系统较简化，主要围绕 int 等基础类型进行检查。

指针、结构体、联合体、复杂数组、多维数组等高级 C 语言特性支持有限或未实现。

LLVM IR 生成为教学化简版本，重点展示四元式到 LLVM IR 的转换思想，并非完整工业级 IR 生成器。

MASM16 汇编生成主要用于展示四元式到目标代码的翻译原则，对复杂运行库和完整链接环境支持有限。

DAG 优化主要针对基本块内部的公共子表达式和局部优化，不进行完整的全局数据流优化。

6.4 心得体会

通过本次课程设计，我对编译原理的理解从书本概念上升到了工程实现层面。以前学习词法分析、语法分析、语义分析和中间代码时，更多是理解定义和算法；而在真正实现一个编译系统后，才更清楚地体会到每个阶段的输入、输出和边界设计都非常重要。一个阶段的数据结构设计不合理，会直接影响后续阶段的实现难度。

本次设计也加深了我对抽象和形式化表达的认识。高级语言中的 if、while、函数调用和表达式，看起来是自然的程序语句，但在编译器内部需要被表示为 Token、AST、符号表和四元式等形式化结构。通过这种转换，程序才能被分析、检查、优化和翻译为目标代码。这让我更深刻地理解了文法、语法树和中间表示的意义。

在工具选择方面，我认识到合适的开发工具能够明显提高实现效率。Python 适合快速实现词法分析、语法分析和数据结构处理，Tkinter 可以较方便地完成可视化界面，unittest 和系统测试脚本能够帮助持续验证功能正确性。通过测试用例驱动检查各个模块，也让我认识到编译器测试不能只依赖正常程序，还必须包含错误用例和边界用例。

本次课程设计还让我认识到文档和目录结构的重要性。随着功能逐渐增加，如果没有清晰的模块划分和输出文件管理，项目会很难维护。通过将核心代码放在 compiler/，将测试用例放在 examples/系统测试用例/，将运行结果放在 outputs/，整个项目结构更加清楚，也更方便报告撰写和功能展示。

同时，我也认识到自己实现的编译器仍然有不少局限。例如，对完整 C 语言特性的支

持还不充分，优化算法主要停留在局部范围，目标代码生成也没有达到真正工业编译器的完整程度。但正是通过这些限制，我更加理解真实编译器的复杂性，也更能体会编译原理课程中各类算法的实际应用价值。

总体来看，本次课程设计达到了课程目标。通过项目实施，我掌握了编译器前端、部分后端、自动机应用和代码优化的基本方法，也学会了如何通过测试验证编译器功能。更重要的是，我认识到编译原理不仅是一门理论课程，它与真实软件工具链、程序语言设计、操作系统和国产基础软件都有密切联系。此次设计提升了我的程序设计能力、抽象建模能力、调试能力和工程文档编写能力。

附录 1：附录文件说明

本课程设计提交的项目目录以完整工程形式组织，主要包括程序源码、测试用例、运行输出、项目说明和展示材料。项目根目录下的主要文件和目录说明如下：

表格 3 项目根目录下的主要文件和目录说明

文件或目录	内容说明
app.py	项目图形化界面入口程序，负责源码编辑、运行、导出、日志识别和结果展示
compiler/	编译器核心功能模块目录，包含词法分析、语法分析、语义分析、中间代码生成、解释执行、LLVM IR、汇编生成、CFG/DAG 优化等模块
examples/	示例程序和系统测试用例目录，用于验证各项功能
outputs/	程序运行后生成的阶段性输出文件目录
tests/	自动化单元测试目录
docs/	实验报告相关说明、测试说明和开发文档
全部测试程序/	课程提供或整理的原始测试程序目录
项目介绍/	项目展示图片和项目介绍材料
README.md	项目说明文件，包含运行方式、功能介绍和输出文件说明
2023 级《编译原理课程设计》任务书（可选版）.docx	课程设计任务书

1. 编译器核心代码目录说明

compiler/ 目录中存放本项目的核心实现代码，各文件功能如下：

表格 4 核心实现代码

文件名	内容说明
lexer.py	词法分析模块，负责生成 Token 序列

文件名	内容说明
parser.py	语法分析模块，负责构造 AST 抽象语法树
semantic.py	语义分析模块，负责符号表管理和语义错误检查
ir.py	中间代码生成模块，负责生成四元式
interpreter.py	中间代码解释器，负责解释执行四元式
assembly.py	MASM16 汇编代码生成模块
llvm_ir.py	LLVM IR 风格代码生成与验证模块
cfg_dag.py	基本块划分、CFG 构建和 DAG 局部优化模块
log_automata.py	日志关键词识别、NFA/DFA 构造展示模块
optimizer.py	中间代码优化模块
target_code.py	目标代码展示生成模块
pipeline.py	编译流水线调度模块
models.py	Token、AST、诊断信息等公共数据结构
source_format.py	源码格式化和自动缩进模块

2. 测试用例目录说明

本项目主要测试用例位于 `examples/系统测试用例/` 目录。该目录按照测试功能划分为多个子目录：

表格 5 测试功能划分

目录名	测试内容说明
01_词法分析/	测试关键字、标识符、常量、字符串、注释、非法字符等词法分析功能
02_语法分析/	测试函数定义、条件语句、循环语句、语法错误恢复和 AST 构造
03_语义分析/	测试符号表、重复声明、未声明变量、函数参数错误、常量赋值错误等
04_中间代码与解释执行/	测试四元式生成、循环、分支、函数调用、数组读写和解释执行
05_MASM16 汇编生成/	测试 MASM16 汇编代码生成、函数调用、栈传参和输入输出过程
06_日志正则自动机/	测试日志关键词识别、正则输入、NFA/DFA 构造和无匹配情况
07_LLVM_IR 生成/	测试 LLVM IR 生成、分支、循环、取模和内部验证
08_CFG 与 DAG	测试基本块划分、CFG 构建、DAG 公共子表达式优化和优化前

目录名	测试内容说明
优化/	后对比
09_GUI 编辑器功能/	测试实时错误标记、诊断表、格式化、自动缩进和语法高亮
run_system_tests.py	系统测试脚本，用于自动验证主要功能
README_系统测试说明.md	系统测试说明文档
系统测试用例总览.md	测试用例总览与预期结果说明

3. 输出文件目录说明

outputs/ 目录用于保存系统运行后生成的各阶段结果文件。主要文件说明如下：

表格 6 保存文件系统

文件名	内容说明
tokens.txt	词法分析结果，保存 Token 序列
ast.txt	语法分析结果，保存 AST 抽象语法树
semantic_errors.txt	语义错误诊断结果
const.txt	常量符号表
var.txt	变量符号表
function.txt	函数符号表
quads.txt	原始四元式中间代码
optimized_quads.txt	普通优化后的四元式
interpreter.txt	中间代码解释执行结果和执行轨迹
llvm_ir.txt	LLVM IR 风格代码输出
llvm_ir.ll	可用于 LLVM 工具链验证的 .ll 文件
llvm_verify.txt	LLVM IR 内部验证结果
assembly.asm	MASM16 汇编代码
target_code.txt	目标代码展示结果
optimized_target_code.txt	优化后目标代码展示结果
basic_blocks.txt	基本块划分结果
cfg.txt	控制流图 CFG 分析结果

文件名	内容说明
dag.txt	DAG 局部优化分析结果
dag_optimized_quads.txt	DAG 优化后的四元式
log_extract.txt	日志关键词提取结果
log_nfa.txt	日志正则对应的 NFA 展示
log_dfa.txt	日志正则对应的 DFA 展示
log_dfa_table.txt	DFA 状态转换表

参考文献

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. Compilers: Principles, Techniques, and Tools[M]. 2nd ed. Boston: Pearson/Addison Wesley, 2007.
- [2] Keith D. Cooper, Linda Torczon. Engineering a Compiler[M]. 2nd ed. Burlington: Morgan Kaufmann, 2011.
- [3] Steven S. Muchnick. Advanced Compiler Design and Implementation[M]. San Francisco: Morgan Kaufmann, 1997.
- [4] Andrew W. Appel, Jens Palsberg. Modern Compiler Implementation in Java[M]. 2nd ed. Cambridge: Cambridge University Press, 2002.
- [5] openEuler 社区. bisheng | openEuler 文档[EB/OL]. https://docs.openeuler.org/zh/docs/20.03_LTS_SP4/docs/thirdparty_migration/bisheng.html, 2026-05-15.
- [6] 华为技术有限公司. 毕昇编译器 2.1.0 用户指南 [EB/OL]. https://repo.huaweicloud.com/kunpeng/archive/compiler/bisheng_compiler/%E6%AF%95%E6%98%87%E7%BC%96%E8%AF%91%E5%99%A82.1.0%E7%94%A8%E6%88%B7%E6%8C%87%E5%8D%97.pdf, 2021-12-30/2026-05-15.
- [7] LLVM Project. LLVM Language Reference Manual[EB/OL]. <https://llvm.org/docs/LangRef.html>, 2026-05-15.
- [8] Python Software Foundation. Python 3.12.13 Documentation[EB/OL]. <https://docs.python.org/3.12/>, 2026-05-15.
- [9] Python Software Foundation. Graphical User Interfaces with Tk[EB/OL]. <https://docs.python.org/3.12/library/tk.html>, 2026-05-15.
- [10] openEuler 社区. 使用 LLVM/Clang 编译[EB/OL]. https://docs.openeuler.org/zh/docs/24.03_LTS_SP2/server/development/application_dev/using_clang_for_compilation.html, 2026-05-15.